Tarefa 17: Tipos Derivados em MPI

Multiplicação Matriz-Vetor com Distribuição por Colunas

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

A tarefa 16 distribuiu a matriz A ($M \times N$) por **linhas** entre os processos MPI. Nesta tarefa, a mesma multiplicação $y = Ax$ é reimplementada distribuindo **colunas**:

- cada processo recebe um bloco de colunas de A e o segmento correspondente de x;
- cada processo calcula uma contribuição parcial para **todos** os elementos de y;
- os vetores parciais são somados no processo 0 com `MPI_Reduce` (`MPI_SUM`).

Como a matriz é armazenada por linhas (*row-major*), um bloco de colunas não é contíguo na memória. Por isso a tarefa usa um **tipo derivado** para descrevê-lo. O objetivo é comparar acesso à memória e desempenho com a versão por linhas.

2 Metodologia

Ferramentas: MPI e supercomputador NPAD.

2.1 Tipo derivado para colunas

Com P processos, cada um fica com `colunas_local` $= \lceil N/P \rceil$ colunas. Como o `MPI_Scatter` exige blocos iguais, a matriz é completada com colunas de zeros até `N_pad` $=$ `colunas_local` $\times P$ (os zeros não alteram o resultado).

O tipo é criado em dois passos (Código 1):

- `MPI_Type_vector`: descreve M blocos de `colunas_local` doubles, com passo de `N_pad` doubles (uma linha inteira). Ou seja, "um bloco de colunas" atravessando todas as linhas.
- `MPI_Type_create_resized`: reduz o *extent* do tipo para `colunas_local` doubles. Sem isso, o `MPI_Scatter` avançaria o tamanho total do tipo entre um processo e outro; com o *resize*, o bloco do processo i começa exatamente na coluna $i \cdot$`colunas_local`.

```c
MPI_Datatype tipo_col, tipo_col_resized;
MPI_Type_vector(M, colunas_local, N_pad, MPI_DOUBLE, &tipo_col);
MPI_Type_create_resized(tipo_col, 0,
                        (MPI_Aint)(colunas_local * sizeof(double)),
                        &tipo_col_resized);
MPI_Type_commit(&tipo_col_resized);

/* envia 1 "bloco de colunas" a cada processo */
MPI_Scatter(A, 1, tipo_col_resized,
            A_local, M * colunas_local, MPI_DOUBLE,
            0, MPI_COMM_WORLD);
```

Código 1: Criação do tipo derivado e uso no Scatter

O processo que recebe os dados os guarda em `A_local` como uma matriz $M \times$ `colunas_local` contígua. O segmento de x é enviado com um segundo `MPI_Scatter` e o resultado é obtido com:

```c
MPI_Reduce(y_parcial, y, M, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

Código 2: Redução dos vetores parciais

2.2 Experimentos

- Matrizes quadradas ($M = N$) de 1024 a 16384.
- Casos *intra* (processos no mesmo nó, 1 a 8 processos) e *inter* (processos em nós diferentes, 2 a 16 processos).
- Tempo **total**: distribuição, cálculo e coleta/redução. Tempo de **cálculo**: apenas o laço de multiplicação. Em ambos, vale o maior tempo entre os processos. Chamamos de *comunicação* a diferença entre os dois.
- A versão por linhas (tarefa 16) foi executada nas mesmas condições.

3 Resultados

A Tabela 1 mostra o maior caso ($N = 16384$). A Tabela 2 traz o *speedup* do tempo total em relação a 1 processo. A Tabela 3 mostra como a diferença entre as versões cresce com o tamanho do problema.

4 Análise

4.1 Diferenças de acesso à memória

- **Linhas:** no processo 0, o bloco de cada processo já é contíguo na memória, então o `Scatter` é uma cópia sequencial. No cálculo, cada processo percorre `A_local` sequencialmente, reutiliza x inteiro (que cabe em cache) e escreve cada `y_local[i]` uma única vez.

Tabela 1: Tempos para $N = 16384$ (segundos). Razão = colunas / linhas (tempo total).

Caso	P	Total linhas	Total colunas	Razão	Cálc. linhas	Cálc. colunas
intra	1	0,4669	0,5108	1,09	0,2690	0,2717
intra	2	0,3515	0,5743	1,63	0,1358	0,1360
intra	4	0,3230	1,0899	3,37	0,0687	0,0678
intra	8	0,3022	1,0867	3,60	0,0348	0,0347
inter	2	0,3451	0,4004	1,16	0,1356	0,1347
inter	4	0,3124	0,4407	1,41	0,0701	0,0675
inter	8	0,2904	0,7304	2,52	0,0344	0,0343
inter	16	0,1310	0,5588	4,27	0,0179	0,0180

Tabela 2: *Speedup* do tempo total para $N = 16384$ (base: 1 processo da respectiva versão).

Caso	P	Linhas	Colunas
intra	2	1,33	0,89
intra	4	1,45	0,47
intra	8	1,54	0,47
inter	2	1,35	1,28
inter	4	1,49	1,16
inter	8	1,61	0,70
inter	16	3,56	0,91

Tabela 3: Razão de tempo total colunas / linhas conforme o tamanho da matriz.

Caso	$N = 1024$	2048	4096	8192	16384
intra, $P = 8$	2,34	2,44	2,34	2,71	3,60
inter, $P = 16$	1,84	2,22	3,46	3,24	4,27

- **Colunas:** o bloco de cada processo está espalhado: são M pedaços pequenos separados por uma linha inteira de A (passo de N doubles). O tipo derivado faz o MPI percorrer a matriz com saltos para montar cada envio, o que tem pior localidade de cache que uma cópia contígua.

- **No cálculo, o acesso é equivalente:** depois do `Scatter`, `A_local` chega contígua, e o laço lê linha a linha, igual à versão anterior. Os tempos de cálculo das duas versões são praticamente iguais (Tabela 1); o custo do acesso espaçado aparece na **distribuição**, não no laço.

- **Saída:** cada processo escreve um `y_parcial` de tamanho M completo (na versão por linhas, só M/P elementos).

4.2 Desempenho em relação à versão por linhas

- **Cálculo escala igual nas duas versões:** o *speedup* do tempo de cálculo fica próximo de P (cerca de 7,8 com 8 processos e 15 com 16, para $N = 16384$). A divisão do trabalho é a mesma.

- **A comunicação é o gargalo da versão por colunas.** Para $N = 16384$ com 8 processos no mesmo nó, a comunicação passa de 0,27 s (linhas) para 1,05 s (colunas); com 16 processos em nós diferentes, de 0,11 s para 0,54 s.

- **Por quê:** na versão por linhas, o `Gather` reúne M elementos no total e o `Bcast` envia x. Na versão por colunas, cada processo contribui para o `Reduce` com um vetor de M elementos, que ainda precisa ser somado. O empacotamento espaçado do `Scatter` soma-se a isso.

- **Efeito no *speedup*:** a versão por linhas chega a 3,56 (inter, 16 processos), enquanto a por colunas não passa de 1,28 e fica abaixo de 1 em vários casos (Tabela 2). Com 4 ou 8 processos no mesmo nó, ela chega a ser mais de duas vezes mais lenta que o programa com 1 processo.

- **A diferença cresce com o problema:** a razão colunas/linhas sobe com N e com P (Tabela 3), chegando a 4,27 vezes. Para $N = 1024$ os tempos são de poucos milissegundos e as medidas são mais ruidosas.

5 Conclusão

- `MPI_Type_vector` com `MPI_Type_create_resized` permite distribuir colunas com um único `MPI_Scatter`, sem cópia manual.

- O laço de cálculo tem o mesmo desempenho que na versão por linhas, pois cada processo trabalha sobre um bloco contíguo.

- A distribuição por colunas é mais lenta, até 4,3 vezes no maior caso, por causa do acesso espaçado na distribuição e da redução de vetores de tamanho M em vez de uma coleta de M/P por processo.

- Para matrizes armazenadas por linhas, a distribuição por linhas é a melhor escolha para $y = Ax$. A distribuição por colunas se justifica quando os dados já estão divididos por colunas.

A Código-Fonte

```c
/**TAREFA 17
 * Multiplicacao matriz-vetor com distribuicao por COLUNAS (tipos derivados MPI)
 *
 * compilacao: mpicc -O3 -o tarefa17 tarefa17.c
 * execucao:  mpirun -np P ./tarefa17 M N [REPS]
 * saida CSV: P,M,N,tempo_total,tempo_calculo
 */

#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <mpi.h>

/**
 * INICIALIZACAO
 */
static void inicializa_matriz(double *A, int M, int N, int ld)
{
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            A[(size_t)i * ld + j] = (double)((i + j) % 10 + 1);
}

static void inicializa_vetor(double *x, int N)
{
    for (int i = 0; i < N; i++)
        x[i] = (double)(i % 5 + 1);
}

/**
 * MAIN
 */
int main(int argc, char **argv)
{
    int rank, P;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &P);

    if (argc < 3) {
        if (rank == 0)
            fprintf(stderr, "Uso: %s <M> <N> [REPS]\n", argv[0]);
        MPI_Finalize();
        return 1;
    }

    int M = atoi(argv[1]);
    int N = atoi(argv[2]);
    int reps = (argc > 3) ? atoi(argv[3]) : 5;

    if (M <= 0 || N <= 0 || reps <= 0) {
        if (rank == 0)
            fprintf(stderr, "Parametros invalidos.\n");
        MPI_Finalize();
        return 1;
```

```
57      }
58
59      /* MPI_Scatter exige blocos iguais -- padding de colunas (zeros) se necessario */
60      int colunas_local = (N + P - 1) / P;
61      int N_pad = colunas_local * P;
62
63      double *A = NULL, *y = NULL, *x = NULL;
64      double *x_local = malloc((size_t)colunas_local * sizeof(double));
65      double *A_local = malloc((size_t)M * colunas_local * sizeof(double));
66      double *y_parcial = malloc((size_t)M * sizeof(double));
67
68      if (!x_local || !A_local || !y_parcial) {
69          fprintf(stderr,
70                  "Processo %d: falha de alocacao (M=%d N=%d P=%d colunas_local=%d, "
71                  "A_local=%.1f MB)\n",
72                  rank, M, N, P, colunas_local,
73                  (double)M * colunas_local * sizeof(double) / 1e6);
74          MPI_Abort(MPI_COMM_WORLD, 1);
75      }
76
77      /**
78       * TIPO DERIVADO:
79       *  - vector: M blocos de 'colunas_local' doubles, com passo N_pad (uma linha
             inteira)
80       *  - resized: extent = colunas_local doubles, para que o i-esimo bloco enviado
81       *    pelo Scatter comece na coluna i*colunas_local
82       */
83      MPI_Datatype tipo_col, tipo_col_resized;
84      MPI_Type_vector(M, colunas_local, N_pad, MPI_DOUBLE, &tipo_col);
85      MPI_Type_create_resized(tipo_col, 0,
86                              (MPI_Aint)(colunas_local * sizeof(double)),
87                              &tipo_col_resized);
88      MPI_Type_commit(&tipo_col_resized);
89
90      if (rank == 0) {
91          A = calloc((size_t)M * N_pad, sizeof(double));
92          x = calloc((size_t)N_pad, sizeof(double));
93          y = malloc((size_t)M * sizeof(double));
94          if (!A || !x || !y) {
95              fprintf(stderr, "Processo 0: falha de alocacao de A/x/y.\n");
96              MPI_Abort(MPI_COMM_WORLD, 1);
97          }
98
99          inicializa_matriz(A, M, N, N_pad);
100         inicializa_vetor(x, N);
101     }
102
103     double soma_total = 0.0, soma_calc = 0.0;
104
105     for (int r = 0; r < reps; r++) {
106         MPI_Barrier(MPI_COMM_WORLD);
107         double t0 = MPI_Wtime();
108
109         /* Distribui blocos de colunas (1 tipo derivado por processo) */
110         MPI_Scatter(A, 1, tipo_col_resized,
111                     A_local, M * colunas_local, MPI_DOUBLE,
112                     0, MPI_COMM_WORLD);
113
```

```
114         /* Distribui o segmento correspondente de x */
115         MPI_Scatter(x, colunas_local, MPI_DOUBLE,
116                     x_local, colunas_local, MPI_DOUBLE,
117                     0, MPI_COMM_WORLD);
118 
119         /* Calculo: contribuicao parcial para TODOS os elementos de y */
120         double tc0 = MPI_Wtime();
121         for (int i = 0; i < M; i++) {
122             double soma = 0.0;
123             for (int j = 0; j < colunas_local; j++)
124                 soma += A_local[(size_t)i * colunas_local + j] * x_local[j];
125             y_parcial[i] = soma;
126         }
127         double tc1 = MPI_Wtime();
128 
129         /* Soma dos vetores parciais no processo 0 */
130         MPI_Reduce(y_parcial, y, M, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
131 
132         double t1 = MPI_Wtime();
133 
134         /* tempo do experimento */
135         double t_total_loc = t1 - t0, t_calc_loc = tc1 - tc0;
136         double t_total, t_calc;
137 
138         MPI_Reduce(&t_total_loc, &t_total, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
139         MPI_Reduce(&t_calc_loc, &t_calc, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
140 
141         if (rank == 0) {
142             soma_total += t_total;
143             soma_calc += t_calc;
144         }
145     }
146 
147     if (rank == 0) {
148         /* verificacao contra calculo serial */
149         double erro = 0.0;
150         for (int i = 0; i < M; i++) {
151             double ref = 0.0;
152             for (int j = 0; j < N; j++)
153                 ref += A[(size_t)i * N_pad + j] * x[j];
154             double d = fabs(ref - y[i]);
155             if (d > erro) erro = d;
156         }
157         if (erro > 1e-6)
158             fprintf(stderr, "ERRO: resultado incorreto (erro max = %g)\n", erro);
159 
160         printf("%d,%d,%d,%.6f,%.6f\n", P, M, N, soma_total / reps, soma_calc / reps);
161 
162         free(A);
163         free(x);
164         free(y);
165     }
166 
167     MPI_Type_free(&tipo_col_resized);
168     MPI_Type_free(&tipo_col);
169     free(x_local);
170     free(A_local);
171     free(y_parcial);
```

```
172
173     MPI_Finalize();
174     return 0;
175 }
```

Código 3: tarefa17.c